WASP Software Engineering Assignment

Talitha Nauta
Department of Automatic Control, Lund University

June 2026

1 Introduction

My research area is automatic control. More specifically I do research on scaled relative graphs (SRGs). The SRG is a novel graphical tool for analysis of operators. The tool was first developed in the field of optimisations where it is used for rigorous convergences proves of both linear and non-linear optimisation algorithms. The aim of my research is to adapt this tool for operators describing dynamical systems and use it for stability and robustness proofs. In this field the SRG can be seen as a generalisation of the Nyquist diagram.

The SRG of an operator

$$\mathbf{A} : \mathcal{H} \to \mathcal{H} \tag{1}$$

is a subset of the extended complex plane $\hat{\mathbb{C}} = \mathbb{C} \cup \{\infty\}$ defined as follows:

$$\mathrm{SRG}\left(\mathbf{A}\right) = \left\{ \frac{\|y_1 - y_2\|}{\|u_1 - u_2\|} \exp\left(\pm i \angle (y_1 - y_2, u_1 - u_2)\right) : u_i \in \mathcal{H}, y_i = \mathbf{A}u_i, u_1 \neq u_2 \right\}. \tag{2}$$

This means that the SRG contains the set of points that describe the possible scaling of the input to the output of the operator as well as the angle between input and output. If we have a SRG of an operator we can therefore derive information about its possible behaviour. If we for example have an operator describing a dynamical system it can be seen in the SRG whether the system is stable or not by checking if the $\{\infty\}$ is included or not. As there are a lot of computational rules for SRGs of interconnected operators, it can also be used for stability analysis of interconnected dynamical systems and feedback loops where we only have SRGs for the individual systems.

A difficult part of working with SRGs is knowing how to compute or over-approximate them, otherwise the theory is useless, and that is what my main current research is about. So far we have been looking into linear closed operators. (A closed operator is an operator for which the graph, set of in and outputs is closed.) In short, the SRG for these operators can be obtained by transforming the SRG to another space in which it is a convex set. This convex can then be bounded by half-planes to get an over- or under-approximation, after which it can be mapped back to the original space where we get the SRG.

This field of research does not have many connections to machine learning and software engineering, as it is mainly based on theoretical mathematics. However, one of the motivations of SRGs is that they in the future can be used to analyse complex systems such as machine learning models, but the research has not come this far yet. Also some code is used and written to understand and validate the mathematical concepts as well as writing software to implement the computation of SRGs so I will make some connections to my user experience in this way.

2 Lecture Principles

2.1 Technical debt

Technical debt in software engineering are long term costs that occur due to the economical or time constraint that force software engineers to work in a high pace [2]. This can lead to for

example entanglement, unstable data dependencies, pipeline jungles and so on. This means that it is cheap to get the code out fast but costly to maintain or update the code later on.

In my research technical debt would mostly mean debt in terms of time. Most of the code I have written for software to compute and draw SRG is written without LLMs tools. This means that I get a good understanding for the code and also for the theory behind the computation from other papers, which my research also builds on. However, already in this case I notice that it can be hard to get back to the code later and understand it if it has not been commented sufficiently. This makes it time consuming when you get back to it and need to understand it again. I have noticed that AI tools can be good at helping to comment the code, even though it has to be reviewed. When it comes to fixing bugs with AI tools I have however noticed that it generally does not manage to fix more complex bugs that occur due to for example numerical errors. In these cases AI tends to create a code jungle that is hard to follow and debug. Also, it tends to create the dead experimental codepaths mentioned in [2], although I have to admit that also happens when coding without AI.

2.2 Testing and Quality Assurance

Testing and quality assurance (QA) is an important field of software engineering that is about verification and validation to ensure that software does the intended task and is correctly implemented.

This relates to my research as SRGs can be used as a tool for proving that optimisation algorithms converge or dynamical systems are stable, which means that is a way of assuring that systems work as intended. Here it is important to notice that there is often a difference between theory and practice. For example if we can prove that an algorithm does converge, it might not due to numerical errors in the code in practice. This means that we need to take some robustness into consideration when using these tools. Similarly, we might not be able to describe our dynamical systems perfectly as operators which leads to incorrect SRG. Therefore it is wise to use over-approximation to include some robustness. This compares to software testing as the software might behave differently under development and out in the real world.

Personally I mostly write code for sanity checks of my mathematical theories. This means that it is of course important to check if the code is correct, but the code itself is already a sort of validation and I usually don't share or publish this code.

3 Guest Lecture Principles

3.1 Requirement specification

Requirement specifications are important in software engineering projects. Especially, if a big group of engineers are coding something together, it is important for them to know what the joint goal is.

When it comes to requirement specifications for the code that I write for my research, especially the code for computing and drawing SRGs, I notice that there is a big difference depending on if I write the code just for myself or also for others. If I only write the code for myself some bugs might be acceptable as long as I understand them and know which faults they give. When showing the code to others this can be highly frustrating. In either case, my code base is so small and I code alone, meaning that there is not really a need to write down the requirements.

3.2 Behavioural Software Engineering

Behavioural software engineering is the field of how human aspects and psychology affect software engineering companies. This can be for example how a company should be organised, how people

get motivated and when people are productive.

As I am not really a software engineer it is hard to directly draw parallels to my own work. However, I think a lot of the things discussed also affect my daily work such as productivity. It is almost impossible to focus on mathematical details 8 hours a day and I need to mix this type of work with, writing, doing administrative task, teaching etc. Generally I think that I am more productive when doing research when I am also teaching during that period. Especially if it is a course I have been teaching before so it does not require much deep thinking in preparation. This means that I will be more concentrated when actually doing research. I do not think that AI is at the point of taking over my teaching role yet, but I can really see how it is a problem if AI takes over the simple tasks, and I do not think it necessarily increases productivity.

4 Data scientists, software engineers, and AI engineers

This section discusses the content of [1] by answering the following question:

What differences do the chapters describe between data scientists and software engineers? Do you agree with this distinction? Why or why not? A data scientist is described like a person who more has a type of research thinking when it comes to coding and handling data. They focus on building models and spend time on gathering and cleaning data. A software engineer would on the other hand more work towards a finished working software that meet requirements given by the user. I think there is a distinction between developing user ready products and building new models as there will always be a try-out phase before getting to a finished product. But as they are both part of the same process maybe the distinction is not that clear.

The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area? As I mostly work with theoretical mathematics I very seldom have a system-wide view on my work. Sometimes looking at the bigger picture is of course important for understanding, but in my research I have and I need to have a very model-centric view.

Do you think data scientists and software engineers will remain distinct roles, or will both need to learn more of each other's skills? Explain. The different roles might still exist, but I think AI tools will simplify for both sides to do the tasks of the others. A software engineer can for example get help from AI to organise data and develop models while a data scientist can get help to create user ready code. This might also make it easier for the two to collaborate as they can get help to understand each other's work.

Where does "AI Engineering" fit in this picture: is it mostly software engineering for AI systems, data science made more systematic, MLOps/model engineering, or something partly distinct? That depends on what the definition of "AI engineering" is, but I think it can come into all parts of the workflow. We have seen that LLMs are very good at solving coding task, meaning they can definitely take over parts of a software engineers job and generate code that meets user requirements. If "AI engineering" also cover organising data and developing models and automating their use it also is very useful for data scientist and MLOps/data engineering.

How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles? I think it will make it easier for both roles to do the task of the other with the help of these tools. As the roles are kind of close, they probably have enough foundation to be able to prompt or setup AI agents in the other area. This means that the boundary will become more vague.

5 Paper analysis

In this section I will discuss two papers from LLM/AI related conferences and connect them to my own research. As this is quite far from my research the connection is not very close, but this is the best I could find. The papers that will be discussed are "LLM Meets Bounded Model Checking: Neuro-symbolic Loop Invariant Inference" by Wu et al [3] and "Towards a Responsible AI Metrics Catalogue: A Collection of Metrics for AI Accountability" by Xia et al. [4].

5.1 LLM Meets Bounded Model Checking

This paper is about finding loop invariants for code loops using LLMs. A loop invariant is a property that holds before the loop starts and remains true after every iteration of the loop. It can be used to formally verify that programs including loops are working as expected. In general, it is difficult to find loop invariants, as they quickly become complex when the size and complexity of the program increase. The paper proposes a framework that combines prompts to LLMs with bounded model checking. The LLM generates candidate loop invariants, while the bounded model checker searches for counterexamples that violate the proposed invariant. If a counterexample is found, it is included in the next prompt, allowing the LLM to refine its proposal. This process is repeated until a valid invariant is found or a predefined limit on runtime or iterations is reached. In the paper, the authors try this with four different LLMs, Llama-3-8B, GPT-3.5-Turbo, GPT-4 and GPT-4-Turbo, where the last gives the best results. The presented framework works in 309 out of 316 benchmark examples.

Formal verification is particularly important for safety-critical software, where failures can have serious consequences. Examples include medical devices, autonomous vehicles, and industrial control systems. By automating part of the verification process, the proposed framework could reduce the effort required to obtain formal guarantees for complex software systems. The counterpart in control theory is to be able to have formal guarantees that a system is stable. Although the domains are different, both approaches seek to construct mathematical abstractions that allow formal reasoning about system behaviour. Loop invariants are used to verify software correctness, while SRGs are used to analyse stability and robustness of dynamical systems.

In an AI-intensive project, the methods proposed in the paper could be used to help the formal verification of the proposed code. If one AI-agent writes code in this project, another AI-agent might be assigned the task of finding loop invariants for loops in the generated code. In this way AI might be used for both code generation and verification. This would increase confidence in the correctness of generated software before deployment. In such a project, SRG-based research could contribute formal methods for analysing the stability and robustness of other subsystems that interact through feedback loops.

An interesting research direction would be to investigate whether LLMs can assist in the discovery of stability certificates, Lyapunov functions, or other mathematical objects used in stability analysis of dynamical systems. Similar to the framework presented in the paper, a verification procedure could be used to reject incorrect candidates and provide feedback to the LLM. Such an approach could potentially accelerate parts of the stability analysis process while still maintaining formal guarantees. However, LLMs are particularly good at processing text- or code-based input, while they are still less capable of constructing mathematical proofs. (Even though it is getting better.) To address this limitation, one possibility would be to use proof-assistant systems such as Lean, which allow mathematical proofs to be expressed and verified as code. This means it gets the text- and code- based structure which LLMs are good at, while it still helps constructing mathematical proofs. I know this has briefly been explored in control theory, but there is still a lot to do.

5.2 Towards a Responsible AI Metrics Catalogue

This paper presents a literature review and catalogue of metrics for assessing accountability in AI systems, with a particular focus on generative AI. Traditional software engineering relies heavily on measurable properties such as test coverage, reliability benchmarks and performance metrics. This becomes significantly harder with the introduction of AI as these are often complex black box models which makes it hard to verify when and how they work. AI tools also bring new challenges because many desirable properties, such as accountability, transparency, and fairness, are difficult to measure directly. A new challenge mentioned in the paper is for example the collaborative nature of AI tools and modification of third party code, which makes it hard to specify who is accountable for the program. Another example is the anthropomorphisation, which means that the systems are perceived as more human-like, while they also have a black box like nature. This can lead to accountability being incorrectly shifted from human actors and organisations to the AI system itself.

The paper presents different types of metrics distributed into several categories. The accountability metrics are organised into the categories responsibility, auditability, and redressability. These address, respectively, who is accountable, what they are accountable for, and how accountability can be enforced or challenged. The metrics are also distributed into three categories. Process metrics, which support procedural integrity, resource metrics, that provide necessary framework and tools, and product metrics, which consider the output of the AI systems. Examples of resource metrics for responsible AI are soft and hard laws. Soft laws can for example be guidelines, governance frameworks and standards of care such as the IEEE 7000 Standard Series. While hard laws are actual laws such as the EU AI Act. Note that many of these metrics are binary, that is to say they are either followed/implemented or not.

Both the Responsible AI Metrics Catalogue and SRG research address the challenge of understanding complex systems through appropriate representations and measurements. The metrics catalogue proposes measurable indicators of accountability, while SRGs provide a mathematical representation of operator behaviour. In both cases, the objective is to replace vague qualitative reasoning with rigorous analysis. In control theory the verification is more on stability and robustness while AI software is more focussed on performance and accountability. In the long term, SRG-based methods could potentially contribute to the development of quantitative metrics for analysing the behaviour, robustness, and reliability of AI models. While such methods would not directly address accountability, they could complement accountability frameworks by providing more rigorous information about system behaviour. This means it could potentially help surrounding the problem that the "big black box" can be used as a scapegoat by providing SRG metric that are easy to interpret. This might also allow for more complex metrics that are not binary, which would lead to better assessments.

The results from this paper allow for better assessment of AI systems which is important when creating big AI-projects. When a company brings a product to the market they must be able to assess if their product is ethical and responsible. As this is a rapidly growing area it is also important that governments have tools to create laws on responsible use of AI and the metrics presented in this paper could help to develop such laws.

6 Evidence, originality, and GenAI-use transparency

The main sources used can be found in the reference list below. [1] is a book for which mainly chapter 1 and 2 were used, the other references [2],[3] and [4] and peer reviewed papers in well known conferences. I consider all these references as trustworthy. The book seems well know with positive reviews and was provided as course literature. The papers are all well cited from highly respected conferences, which gives good confidence that the work can be trusted.

GPT-5.5 has been used to find fitting papers for the paper analysis in section 5 and for improving the text, such as grammar, punctuation and formulations, after first writing a full

draft without AI-tools.

References

[1] C. Kastner. *Machine Learning in Production: From Models to Products*. MIT Press, 2025.

[2] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.

[3] Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Llm meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ASE '24, page 406–417, New York, NY, USA, 2024. Association for Computing Machinery.

[4] Boming Xia, Qinghua Lu, Liming Zhu, Sung Une Lee, Yue Liu, and Zhenchang Xing. Towards a responsible ai metrics catalogue: A collection of metrics for ai accountability. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI*, CAIN '24, page 100–111, New York, NY, USA, 2024. Association for Computing Machinery.